

Practical: 5

Implementation of a knapsack problem using dynamic programming.

25/08/2026

AIM: To Implement the knapsack problem using dynamic programming.

ALGORITHM:

Algorithm for knapsack Using DP.

Steps:-

1. Start
2. Read the no. of items n .
3. Read the maximum capacity of the knapsack W .
4. Read the weight and value of each item.
5. Create DP table $dp[n+1][W+1]$
6. Initialize first row and column with 0.
7. For each item and capacity.
 - * if item weight $\leq$ capacity, choose maximum of including (or) excluding the item.
 - * otherwise, exclude the item.
8. Return $dp[n][W]$.
9. Stop.

CODE:

```
#include<iostream>
using namespace std;

class Knapsack {

private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity
    int dp[10][100]; // dp table

public:

    // ?? Get Input ??
    void getInput() {
        cout << "Enter number of items  : ";
        cin >> n;
        cout << "Enter knapsack capacity : ";
        cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    // ?? Build dp table ??
    void solve() {

        // base case - row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;

        // fill table
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {

                if(w[i] <= j) {
                    // take or not take
                    dp[i][j] = max(
                        v[i] + dp[i-1][j-w[i]],
                        dp[i-1][j]
                    );
                } else {
                    // cannot take item i
                    dp[i][j] = dp[i-1][j];
                }
            }
        }
    }
}
```

```
    }  
  }  
}
```

```
// ?? Print dp table ??
```

```
void printTable() {  
    cout << "\nDP Table:" << endl;  
    cout << "  ";  
    for(int j = 0; j <= W; j++)  
        cout << j << " ";  
    cout << endl;
```

```
    for(int i = 0; i <= n; i++) {  
        cout << "i=" << i << " ";  
        for(int j = 0; j <= W; j++)  
            cout << dp[i][j] << " ";  
        cout << endl;  
    }  
}
```

```
// ?? Find selected items ??
```

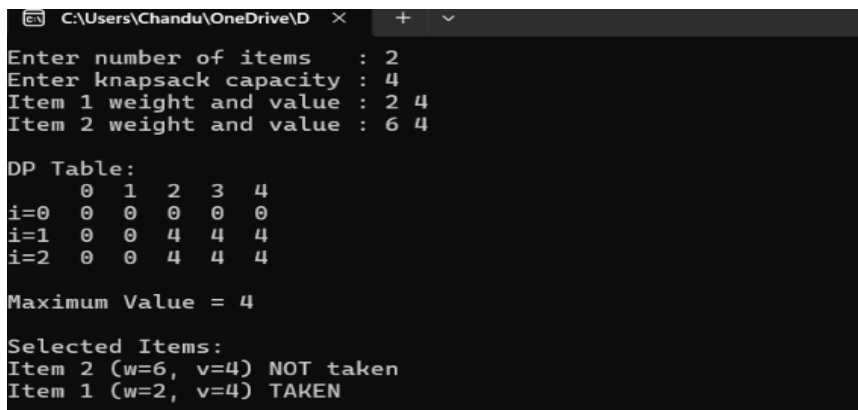
```
void findItems() {  
    cout << "\nSelected Items:" << endl;  
    int j = W;  
    for(int i = n; i >= 1; i--) {  
        if(dp[i][j] != dp[i-1][j]) {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") TAKEN " << endl;  
            j = j - w[i];  
        } else {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") NOT taken " << endl;  
        }  
    }  
}
```

```
// ?? Show result ??
```

```
void showResult() {  
    cout << "\nMaximum Value = "  
        << dp[n][W] << endl;  
}  
};
```

```
// ?? MAIN ??  
int main() {  
  
    Knapsack k;  
  
    k.getInput();  
    k.solve();  
    k.printTable();  
    k.showResult();  
    k.findItems();  
  
    return 0;  
}
```

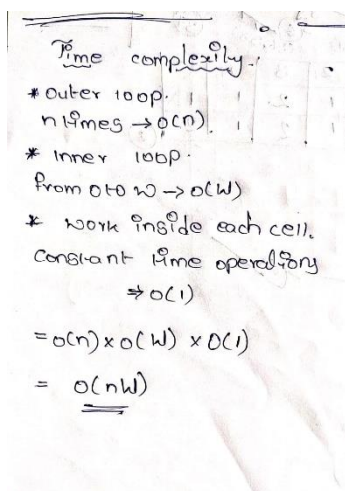
OUTPUT:



```
C:\Users\Chandu\OneDrive\D > .\DAA.exe  
Enter number of items : 2  
Enter knapsack capacity : 4  
Item 1 weight and value : 2 4  
Item 2 weight and value : 6 4  
  
DP Table:  
   0  1  2  3  4  
i=0  0  0  0  0  0  
i=1  0  0  4  4  4  
i=2  0  0  4  4  4  
  
Maximum Value = 4  
  
Selected Items:  
Item 2 (w=6, v=4) NOT taken  
Item 1 (w=2, v=4) TAKEN
```

TIMECOMPLEXITY:

BEST CASE:

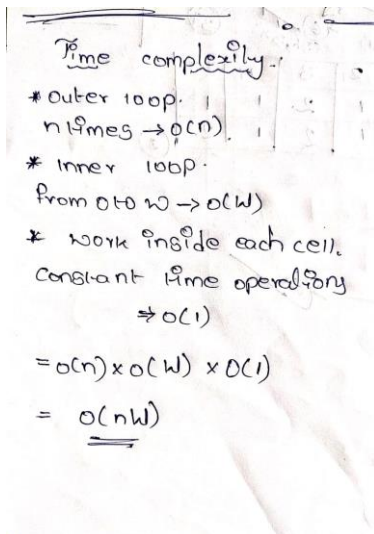


Time complexity:

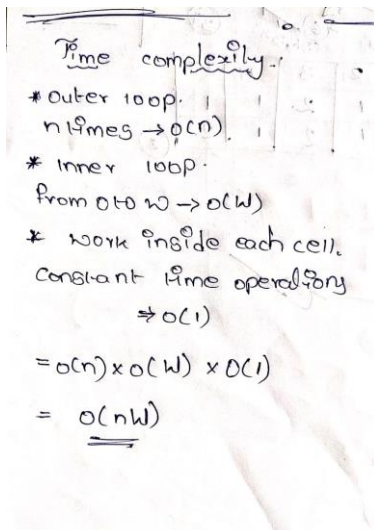
- * Outer loop:
 n times $\rightarrow O(n)$
- * Inner loop:
From 0 to $w \rightarrow O(W)$
- * Work inside each cell:
Constant time operation
 $\Rightarrow O(1)$

$$= O(n) \times O(W) \times O(1)$$
$$= \underline{O(nW)}$$

AVERAGE CASE:



WORST CASE:



CONCLUSION:

The 0/1 Knapsack problem can be solved efficiently using Dynamic Programming. It finds the maximum value by choosing the best items within the given capacity.